

Introduction to Text Representation in NLP

Introduction to Text Representation in NLP

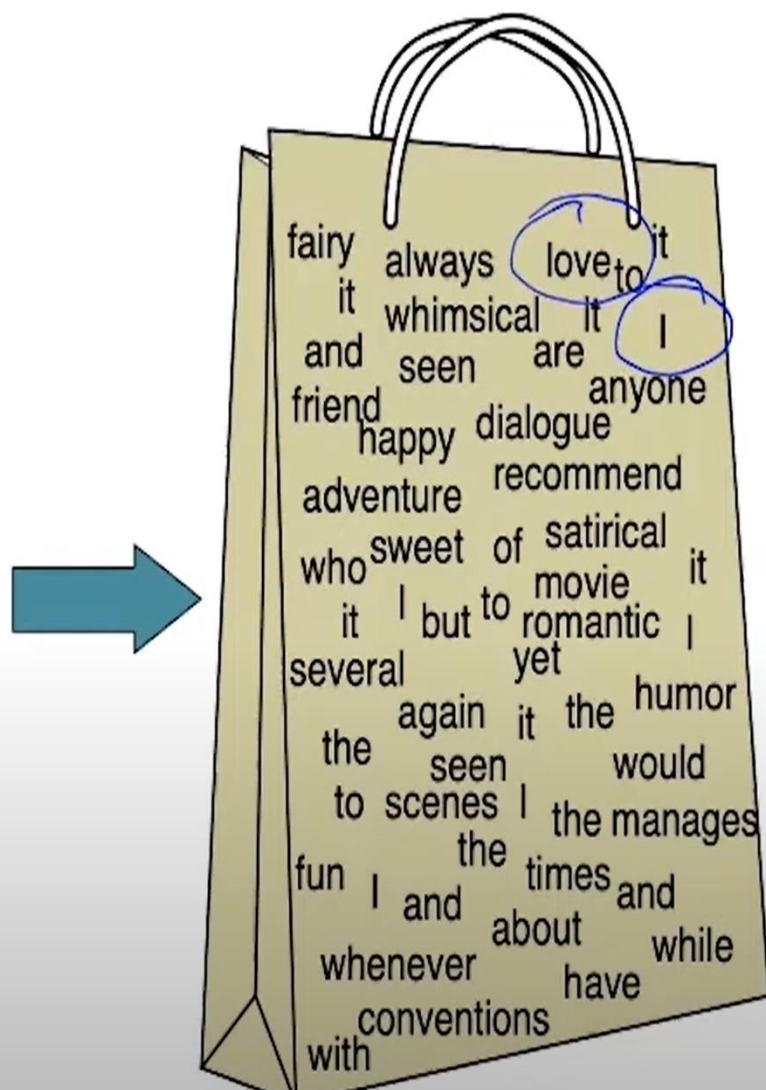
- Text data is **unstructured**, unlike numerical data.
- For many application, Text data needs to be transformed into **numerical representations (vectors)** for tasks like:
 - Text Classification (e.g., spam detection), Sentiment Analysis, Named Entity Recognition (NER), Machine Translation, Text Generation
 - **Similarity-based Tasks**: Tasks like text similarity, clustering, or search/retrieval benefit from vector representations.
- Converting text into a format that machines can understand is essential.
- Common vector representations include **BoW, TF-IDF, Word2Vec, GloVe, fastText, and recent transformers-based embeddings (e.g., BERT)**.

What is Bag of Words (BoW)?

- **Definition:**
 - BoW is a simple text representation technique in NLP.
 - It represents text as a collection of words, disregarding grammar, order, and structure.
- **Key Characteristics:**
 - Treats each document as a "bag" of its words.
 - Does not account for word order (e.g., "dog bites man" is the same as "man bites dog").
 - Focuses on the frequency of words in the text.

The Bag of Words Representation

I love this movie! It's sweet,
but with satirical humor. The
dialogue is great and the
adventure scenes are fun...
It manages to be whimsical
and romantic while laughing
at the conventions of the
fairy tale genre. I would
recommend it to just about
anyone. I've seen it several
times, and I'm always happy
to see it again whenever I
have a friend who hasn't
seen it yet!



it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1



The bag of words representation

$Y($

great	2
love	2
recommend	1
laugh	1
happy	1
...	...

$) = C$




Contd..

- **Advantages:**
 - Simple and fast to compute.
 - Simplicity: Easy to implement and understand.
 - Effective for Small Texts: Works well for small datasets or in basic applications.

Limitations of Bag of Words

- **Ignores word order:** Meaning can be lost (e.g., “The man bites the dog” vs. “The dog bites the man”).
- **Ignores Semantics and context:** Cannot capture the meaning of words (e.g., “great” and “awesome” are treated as different).
- **High Dimensionality:** For large vocabularies, the vector size increases, leading to computational inefficiency (curse of dimensionality).
- **Sparse Representation:** Lots of zeros in the resulting vectors.
 - BoW results in sparse matrices (many 0s) as most documents don’t contain every word in the vocabulary.

How Bag of Words Works

- Common Steps:
 - Tokenization: Split text into individual words (tokens).
 - Removing Stop Words and Punctuation
 - Vocabulary Creation: Create a list of all unique words in the corpus.
 - Word Frequency Count: Count the occurrence of each word in each document.
 - Vector Representation: Represent each document as a vector of word frequencies.

Bag of Words to create a Bag of Words representation

Document 1: "John likes to watch movies. Mary likes movies too."

Document 2: "John also likes to watch football games."

Step 1: Tokenization

Break each document into individual words (tokens).

Document 1 Tokens: ["John", "likes", "to", "watch", "movies", ".", "Mary", "likes", "movies", "too", "."]

Document 2 Tokens: ["John", "also", "likes", "to", "watch", "football", "games", "."]

contd

Step 2: Removing Stop Words and Punctuation

Remove common words like "to," "also," "too," and punctuation marks.

Document 1 Processed: ["John", "likes", "watch", "movies", "Mary", "likes", "movies"]

Document 2 Processed: ["John", "likes", "watch", "football", "games"]

Step 3: Create Vocabulary

List all unique words from both documents.

Vocabulary: ["John", "likes", "watch", "movies", "Mary", "football", "games"]

Step 4: Word Frequency Count

Count how often each word from the vocabulary appears in each document.

Word	Document 1 Count	Document 2 Count
John	1	1
likes	2	1
watch	1	1
movies	2	0
Mary	1	0
football	0	1
games	0	1

contd

Step 5: Vector Representation

Convert each document into a vector, where each dimension corresponds to a word from the vocabulary, and the values represent the word counts.

Document 1 Vector: [1, 2, 1, 2, 1, 0, 0]

Document 2 Vector: [1, 1, 1, 0, 0, 1, 1]

Step 6: Resulting Bag of Words Representation

Document 1 BoW Vector: [1, 2, 1, 2, 1, 0, 0]

Document 2 BoW Vector: [1, 1, 1, 0, 0, 1, 1]

Enhancements to Bag of Words

- **TF-IDF (Term Frequency-Inverse Document Frequency):** Weighs words based on importance across documents.
- **Word Embeddings (Word2Vec, GloVe):** Learns dense representations of words that capture semantics.

tf-idf weighting

The intuition is that a word appearing 100 times in a document doesn't make that word 100 times more likely to be relevant to the meaning of the document. Because we can't take the log of 0, we normally add 1 to the count:

$$\text{tf}_{t,d} = \text{count}(t,d)$$

$$\text{tf}_{t,d} = \log_{10}(\text{count}(t,d) + 1)$$

If we use log weighting, terms which occur 0 times in a document would have $\text{tf} = \log_{10}(1) = 0$, 10 times in a document $\text{tf} = \log_{10}(11) = 1.04$, 100 times $\text{tf} = \log_{10}(101) = 2.004$, 1000 times $\text{tf} = 3.00044$, and so on.

Contd..

IDF Calculation

$$\text{idf}_t = \log_{10} \left(\frac{N}{\text{df}_t} \right)$$

- N is the total number of documents in the corpus.
- df_t is the number of documents in which term t appears.

TF-IDF Calculation:

$$\text{TF-IDF}_{t,d} = \text{tf}_{t,d} \times \text{idf}_t$$

Step-by-Step Example:

We will calculate **TF-IDF** for the following documents:

1. **D1**: "the cat sat on the mat"
2. **D2**: "the dog sat on the log"
3. **D3**: "the cat chased the dog"

Our goal is to compute the **TF-IDF** for each word in **D1** using the formulas:

- **TF formula:**

$$\text{TF}_{t,d} = \log_{10}(\text{count}(t, d) + 1)$$

- **IDF formula:**

$$\text{IDF}_t = \log_{10} \left(\frac{N}{\text{df}_t} \right)$$

where N is the total number of documents, and df_t is the number of documents that contain the term t .

Step 1: Calculate TF for D1

For D1 ("the cat sat on the mat"), let's calculate the term frequency (TF) for each word:

- **the** appears 2 times in D1:

$$\text{TF}(\text{the}, D1) = \log_{10}(2 + 1) = \log_{10}(3) \approx 0.4771$$

- **cat** appears 1 time:

$$\text{TF}(\text{cat}, D1) = \log_{10}(1 + 1) = \log_{10}(2) \approx 0.3010$$

- **sat** appears 1 time:

$$\text{TF}(\text{sat}, D1) = \log_{10}(1 + 1) = \log_{10}(2) \approx 0.3010$$

- **on** appears 1 time:

$$\text{TF}(\text{on}, D1) = \log_{10}(1 + 1) = \log_{10}(2) \approx 0.3010$$

- **mat** appears 1 time:

$$\text{TF}(\text{mat}, D1) = \log_{10}(1 + 1) = \log_{10}(2) \approx 0.3010$$

Step 2: Calculate IDF for Each Word in D1

We now calculate the **IDF** for each term across the three documents. The total number of documents $N = 3$.

- **the** appears in all three documents (D1, D2, D3):

$$\text{IDF}(\text{the}) = \log_{10} \left(\frac{3}{3} \right) = \log_{10}(1) = 0$$

- **cat** appears in two documents (D1, D3):

$$\text{IDF}(\text{cat}) = \log_{10} \left(\frac{3}{2} \right) = \log_{10}(1.5) \approx 0.1761$$

- **sat** appears in two documents (D1, D2):

$$\text{IDF}(\text{sat}) = \log_{10} \left(\frac{3}{2} \right) = \log_{10}(1.5) \approx 0.1761$$

- **on** appears in two documents (D1, D2):

$$\text{IDF}(\text{on}) = \log_{10} \left(\frac{3}{2} \right) = \log_{10}(1.5) \approx 0.1761$$

- **mat** appears in only one document (D1):

$$\text{IDF}(\text{mat}) = \log_{10} \left(\frac{3}{1} \right) = \log_{10}(3) \approx 0.4771$$

Step 3: Calculate TF-IDF for Each Word in D1

To compute **TF-IDF**, multiply the TF by the corresponding IDF for each word.

- the:

$$\text{TF-IDF}(\text{the}, D1) = 0.4771 \times 0 = 0$$

- cat:

$$\text{TF-IDF}(\text{cat}, D1) = 0.3010 \times 0.1761 \approx 0.0530$$

- sat:

$$\text{TF-IDF}(\text{sat}, D1) = 0.3010 \times 0.1761 \approx 0.0530$$

- on:

$$\text{TF-IDF}(\text{on}, D1) = 0.3010 \times 0.1761 \approx 0.0530$$

- mat:

$$\text{TF-IDF}(\text{mat}, D1) = 0.3010 \times 0.4771 \approx 0.1435$$

Final TF-IDF Scores for D1:

Word	TF	IDF	TF-IDF
the	0.4771	0	0
cat	0.3010	0.1761	0.0530
sat	0.3010	0.1761	0.0530
on	0.3010	0.1761	0.0530
mat	0.3010	0.4771	0.1435

Conclusion:

- The word "**mat**" has the highest TF-IDF score in **D1** because it is relatively unique, appearing only in one document.
- The words "**cat**", "**sat**", and "**on**" have moderate scores as they appear in more than one document.
- The word "**the**" has a TF-IDF score of zero because it appears in all documents, meaning it doesn't provide much distinctive information.

TF-IDF for D2 ("the dog sat on the log"):

Word	TF	IDF	TF-IDF
the	0.4771	0	0
dog	0.3010	0.1761	0.0530
sat	0.3010	0.1761	0.0530
on	0.3010	0.1761	0.0530
log	0.3010	0.4771	0.1435

TF-IDF for D3 ("the cat chased the dog"):

Word	TF	IDF	TF-IDF
the	0.4771	0	0
cat	0.3010	0.1761	0.0530
chased	0.3010	0.4771	0.1435
dog	0.3010	0.1761	0.0530

Final TF-IDF Matrix:

The final matrix for all three documents and their terms is:

Word	D1 (TF-IDF)	D2 (TF-IDF)	D3 (TF-IDF)
the	0	0	0
cat	0.0530	0	0.0530
sat	0.0530	0.0530	0
on	0.0530	0.0530	0
mat	0.1435	0	0
dog	0	0.0530	0.0530
log	0	0.1435	0
chased	0	0	0.1435

Conclusion:

- **Common terms** like "the" have a TF-IDF of 0 across all documents because they appear in every document, providing no distinguishing information.
- Terms like "**mat**", "**log**", and "**chased**" have the highest TF-IDF values in their respective documents because they are more unique.